



PNGtoCCSB

User Manual

Creating sprites for the BasTo6809 compiler

PNGtoCCSB version 1.11
BasTo6809 compiler version 5.46
Glen Hewlett - August 2026

Contents

What PNGtoCCSB does

Five-minute sprite tutorial

Preparing good source artwork

Choosing the GMODE

Animated sprites

Sprite update patterns

Background handling options

CoCo 3 palette workflow

Command-line option reference

Background-image output

Troubleshooting

Quick-reference checklist

Command examples

What PNGtoCCSB does

PNGtoCCSB converts a modern PNG picture into 6809 assembly code that the BasTo6809 compiler can include as a sprite. It performs the difficult parts automatically:

- matches the PNG colours to the selected CoCo graphics mode;
- preserves transparent areas;
- packs pixels into the CoCo screen-memory format;
- creates the drawing and background-restoration routines; and
- creates all alignment versions needed when a sprite begins between packed-byte boundaries.

The usual workflow is:

1. Draw or export a transparent RGBA PNG.
2. Run PNGtoCCSB for the same GMODE your BASIC program uses.
3. Load the generated `.asm` file with `SPRITE_LOAD`.
4. Set up two matching graphics pages.
5. Draw the sprite with the `SPRITE` commands and update it with `WAIT VBL`.

PNGtoCCSB can also export full-screen background images. That feature is separate from the compiled-sprite workflow and is described in [Background-image output](#).

The most important rule: the `-g` value used by PNGtoCCSB must be the exact GMODE used by the BASIC program. A sprite generated for one screen format cannot safely be used in another.

Five-minute sprite tutorial

1. Prepare the PNG

Create a 32-bit RGBA PNG named `HERO.png`. Transparent pixels become holes through which the existing background remains visible. Opaque black is a real drawn colour; it is not treated as transparency.

For reliable assembly labels, use a simple filename:

- begin with a letter;
- use letters, numbers, and underscores only; and
- avoid spaces, hyphens, extra periods, and punctuation.

Good names include `HERO.png`, `PLAYER_1.png`, and `RED_DRAGON.png`.

2. Convert it

Open Terminal in the compiler folder and enter:

```
./PNGtoCCSB -g15 -s0 HERO.png
```

On Windows, use the supplied Windows executable name without the leading `./`.

This command means:

- `-g15` - generate a sprite for GMODE 15;
- `-s0` - use the colour set selected by `SCREEN 1,0`; and
- `HERO.png` - read this input picture.

PNGtoCCSB creates `HERO.asm` in the current working folder. It also reports the generated width, height, colour count, and number of frames. Use those generated dimensions when checking screen limits because semigraphics conversion can change the sprite's size.

All options must appear **before** the PNG filename:

```
PNGtoCCSB [options] InputFile.png
```

If the input path contains spaces, quote it:

```
./PNGtoCCSB -g15 -s0 "Artwork/Hero Images/HERO.png"
```

The output is still named from the PNG's base name and is written to the current working folder.

3. Load it in BASIC

Add this compile-time statement to the BASIC source:

```
Sprite_Load "HERO.asm", 0
```

The second argument is the sprite number. Valid sprite numbers are 0 through 31.

The compiler opens sprite files relative to its working folder. Keeping the generated `.asm` beside the compiler is simplest. A relative subfolder also works when the path is relative to that folder:

```
Sprite_Load "Sprites/HERO.asm", 0
```

4. Prepare two graphics pages

CoCo 1/2 sprites use double buffering. Reserve page 1, return to page 0, draw the complete background, and copy it to page 1:

```
Gmode 15, 1
Gmode 15, 0
GCLs 0
Screen 1, 0

' Draw the background here.

GCopy 0, 1
```

Do not omit the first `GMODE 15,1`. Even when page 0 is initially displayed, the sprite system updates both pages and therefore needs both pages reserved.

5. Draw and update the sprite

This complete example places sprite 0 on the screen:

```

'compileoptions -s16 -i

Sprite_Load "HERO.asm", 0

Dim X, Y As Integer

Gmode 15, 1
Gmode 15, 0
Gcls 0
Screen 1, 0

' Draw the finished background before copying it.
GCopy 0, 1

X = 20
Y = 20

Do
    Sprite Locate 0, X, Y
    Sprite Backup 0
    Sprite Show 0, 0

    Wait VBL

    Sprite Erase 0
Loop

```

`SPRITE BACKUP`, `SPRITE SHOW`, and `SPRITE ERASE` add work to the sprite queue. `WAIT VBL` performs the queued work while synchronizing it with the display. The `ERASE` after `WAIT VBL` is queued for the following update.

Keep the entire sprite on the screen. The runtime does not clip sprites at the edges; drawing partly outside the screen can overwrite unrelated memory and crash the program.

Preparing good source artwork

RGBA and transparency

Use a 32-bit RGBA PNG. PNGtoCCSB treats alpha values below 127 as fully transparent and values of 127 or greater as fully opaque. There is no partial transparency or alpha blending.

For clean pixel art:

- use hard-edged transparency rather than soft antialiased edges;
- draw at the approximate size needed on the CoCo;
- avoid more colours than the destination mode can display; and
- preview the result on the intended CoCo or emulator video configuration.

PNGtoCCSB chooses the nearest available hardware colours, so a photographic image or a sprite with many similar shades will be simplified.

File and label names

The PNG base name is also used inside the assembly file for labels. Renaming only the generated `.asm` file does not rename those labels. Regenerate the output from a correctly named PNG instead.

Recommended:

```
PLAYER.png  
PLAYER_WALK.png  
ALIEN_03.png
```

Avoid:

```
3rd-player.png  
player walk.png  
player.v2.png
```

Size and edge safety

The source PNG dimensions are not always the final CoCo dimensions. GMODE 0 through GMODE 8 apply cell and aspect-ratio conversions. Always read the dimensions printed by PNGtoCCSB and leave enough room that:

```
X + generated width <= screen width  
Y + generated height <= screen height
```

For packed-pixel animation sheets, make each frame naturally byte-aligned when possible:

Destination format	Recommended width of each frame
2 colours	multiple of 8 pixels
4 colours	multiple of 4 pixels
16 colours	multiple of 2 pixels

This prevents padding at the end of the complete strip from changing the expected per-frame layout.

Choosing the GMODE

The command-line GMODE must match the BASIC `GMODE` statement exactly. The following table describes the sprite-capable families without repeating every resolution listed in the main compiler manual.

GMODE	Compiled sprites	Coordinates and important notes
0-1	Yes	32 x 16 whole display cells. PNGtoCCSB applies 3:2 horizontal correction. There is no sub-cell movement.
2	Yes	SG4 cells. Best colour fidelity at even X and even Y. Corrected width is at most 32 cells; source height is at most 16 rows.
3, 5-8	Yes	Two horizontal subpixels share each colour cell. Best fidelity at even X. PNGtoCCSB corrects vertical proportions automatically; source width is at most 32 pixels.
4	Yes	SG6 cells. Best at even X and a Y divisible by 3. Requires an original MC6847; the CoCo 2B MC6847T1 does not display true SG6.
9, 11, 13, 15	Yes	Packed 4-colour pixels. <code>-s0</code> or <code>-s1</code> must match the BASIC <code>SCREEN</code> colour set.
10, 12, 14, 16	Yes	Packed 2-colour pixels. Source pixels are reduced to black or nonblack.
17	Do not use	DMA placeholder mode; the compiler's GMODE reference identifies it as unusable.
18	Yes	One-bit NTSC artifact-colour mode. Appearance depends on composite phase and the display.
100-159	Yes	CoCo 3 packed 2-, 4-, or 16-colour modes. They use one shared <code>CoCo3_Palette.asm</code> and the current sprite system requires a 2 MB CoCo 3.
160-165	No compiled-sprite output	Special 256-colour NTSC-composite image modes. These are background/image targets, not current <code>SPRITE_LOAD</code> targets.

The specialized GMODE 0-8 output records the required mode in the generated file and the compiler checks it. Older packed-pixel output does not yet contain that automatic marker, so manually matching `g#` to `GMODE #` remains essential for every mode.

CoCo 1/2 colour sets

`-s0` is the default. Use `-s1` only when the program selects the second hardware colour set. For example:

```
./PNGtoCCSB -g15 -s1 FIREBALL.png
```

```
Gmode 15, 1
Gmode 15, 0
Screen 1, 1
Sprite_Load "FIREBALL.asm", 0
```

The option matters for GMODE 1, GMODE 4, and the normal CoCo 1/2 four-colour packed modes. Actual analogue shades can vary with the MC6847 revision, emulator, composite processing, and display.

Semigraphics alignment summary

PNGtoCCSB generates shifted drawing routines, so odd positions are supported. Cell-aligned positions normally preserve the source colours most exactly:

Mode	Best-aligned position
GMODE 2	even X, even Y
GMODE 4	even X, Y divisible by 3
GMODE 3 and 5-8	even X

GMODE 0 and GMODE 1 use whole 32 x 16 cells and cannot move inside a cell.

Animated sprites

Place every frame in one horizontal strip. Every frame must have the same width and height, and the total image width must divide evenly by the number of frames.

For an eight-frame strip:

```
./PNGtoCCSB -g15 -s0 -a8 WOLF_WALK.png
```

`-a8` means that the image contains eight frames; it does not mean that 8 is the highest frame number. PNGtoCCSB supports up to 32 frames in one sheet.

Repeat the frame count in `SPRITE_LOAD` :

```
Sprite_Load "WOLF_WALK.asm", 2, 8
```

The valid frame numbers are 0 through 7:

```
Sprite Locate 2, WolfX, WolfY  
Sprite Backup 2  
Sprite Show 2, Frame
```

Keep `Frame` in range. The runtime does not currently check frame bounds; an invalid frame can jump outside the sprite's routine table.

`-awN` is an advanced legacy walk-cycle/phase option for GMODE 18 only. Use ordinary `-aN` for all normal animation sheets.

Sprite update patterns

Moving one sprite

Update the coordinates, queue the draw operations, wait for vertical blank, and then queue the erase for the following update:

```
SpriteUpdate:
    Sprite Locate 0, PlayerX, PlayerY
    Sprite Backup 0
    Sprite Show 0, PlayerFrame
    Wait VBL
    Sprite Erase 0
Return
```

Call the routine once for every new frame:

```
GoSub SpriteUpdate
```

Overlapping sprites

Draw sprites from back to front and erase them in reverse order:

```
Sprite Locate 0, X0, Y0
Sprite Backup 0
Sprite Show 0, F0

Sprite Locate 1, X1, Y1
Sprite Backup 1
Sprite Show 1, F1

Wait VBL

Sprite Erase 1
Sprite Erase 0
```

Reversing the erase order preserves the saved background layers when sprites overlap.

Avoiding flicker

Use `WAIT VBL` for every visible sprite update. Do not erase a sprite, wait, and then draw it again in a later frame; that deliberately displays one frame without the sprite. Also make sure both graphics pages begin with the same background by calling `GCOPY 0,1` after the background is complete.

Background handling options

The `-b` option controls what happens behind a compiled sprite.

Option	Behaviour	When to use it
<code>-b1</code>	Saves and restores the original background. This is the default.	Normal moving sprites over detailed backgrounds.
<code>-b0</code>	Does not save or restore the background. The sprite is destructive.	Maximum speed when the program redraws the complete scene itself.
<code>-b2 -pN</code>	Restores one constant packed-byte pattern instead of saved pixels.	A uniform background whose exact memory byte is known.

Example:

```
./PNGtoCCSB -g15 -s0 -b2 -p0 BULLET.png
```

`-pN` is a raw byte value from 0 through 255, not a BASIC colour number. Some useful repeated-colour patterns are:

Packed format	Repeated colour index	Byte value
2-colour	0 / 1	0 / 255
4-colour	0 / 1 / 2 / 3	0 / 85 / 170 / 255
16-colour	index N	N x 17

With `-b0`, `SPRITE BACKUP` and `SPRITE ERASE` cannot recover covered pixels. With `-b2`, `SPRITE ERASE` writes the selected pattern rather than the original picture.

CoCo 3 palette workflow

GMODE 100 through GMODE 159 use `CoCo3_Palette.asm`. Every sprite in one program must be converted against the same palette.

Create the shared palette once

Choose artwork representative of the project and convert it with `-makepal`:

```
./PNGtoCCSB -g136 -makepal PLAYER.png
```

This creates or replaces `CoCo3_Palette.asm`, selects up to 16 frequently used quantized colours, and converts `PLAYER.asm` against that palette.

Reuse it for every other sprite

Leave off `-makepal`:

```
./PNGtoCCSB -g136 ENEMY.png  
./PNGtoCCSB -g136 EXPLOSION.png
```

PNGtoCCSB automatically reads the existing `CoCo3_Palette.asm`. The stale `-usepal` name shown by older help text is not a real option; using the existing palette is already the default.

Keep the palette file with the generated sprite files and do not regenerate it halfway through a project unless every sprite is converted again. The compiler includes and loads the shared palette when it encounters the first CoCo 3 `SPRITE_LOAD`.

Horizontally scrolling playfields

Use `-scroll` only for sprites drawn on a CoCo 3 horizontally scrolling playfield whose rows have the required 256-byte stride:

```
./PNGtoCCSB -g136 -scroll PLAYER.png
```

Do not use `-scroll` for an ordinary graphics screen, a non-scrolling background, a vertical-only scrolling layout, or PLAYFIELD 1. It is intended for the compiler's horizontally scrolling PLAYFIELD 2 through PLAYFIELD 5 layouts.

Command-line option reference

All option letters are case-insensitive. Attach each value directly to its option: write `-g15` , not `-g 15` .

Required and common options

Option	Default	Meaning
<code>-gN</code>	required	Destination GMODE. It must match the BASIC program exactly.
<code>-s0</code> or <code>-s1</code>	<code>-s0</code>	CoCo 1/2 hardware colour set; match <code>SCREEN 1,0</code> or <code>SCREEN 1,1</code> .
<code>-vN</code>	<code>-v0</code>	Diagnostic detail from 0 through 3.

Sprite options

Option	Default	Meaning
<code>-aN</code>	<code>-a1</code>	Number of equal-width animation frames in one horizontal strip; maximum 32.
<code>-awN</code>	off	Advanced GMODE 18 walk-cycle/phase format.
<code>-oN</code>	<code>-o0</code>	Skip N pixels from the left edge of the complete PNG or animation sheet.
<code>-b0</code>		Destructive sprite; no background backup or restoration.
<code>-b1</code>	selected	Save and restore the background.
<code>-b2</code>		Restore a constant byte pattern; requires <code>-pN</code> .
<code>-pN</code>	none	Raw restore byte from 0 through 255 for <code>-b2</code> .
<code>-scroll</code>	off	Use the special 256-byte row stride for CoCo 3 horizontal playfields.
<code>-makepal</code>	off	Create/replace <code>CoCo3_Palette.asm</code> , then use it for this conversion.

`-oN` crops once from the left of the entire source, not once from every animation frame. It is intended for advanced sheet preparation; cropping the PNG in an image editor is usually clearer.

Background-image options

Option	Output intended by the program
<code>-c1</code>	LOADM-format <code>.BIN</code> with a load address of zero.
<code>-c2</code>	Headerless packed <code>.RAW</code> data.
<code>-c3</code>	Assembly <code>.ASM</code> containing <code>FCB</code> data.
<code>-c4</code>	Banked CoCo 3 256-colour composite LOADM <code>.BIN</code> .

Option	Output intended by the program
<code>-c5</code>	CoCo 3 256-colour composite <code>.RAW</code> .
<code>-c6</code>	CoCo 3 256-colour composite <code>FCB</code> assembly.
<code>-b1kN</code>	First 8K CoCo 3 memory block for <code>-c4</code> ; decimal N, default 0.
<code>-dN</code>	Dither for <code>-c4</code> through <code>-c6</code> : 0 none, 1 Floyd-Steinberg, 2 blue noise, 3 ordered.
<code>-x0</code> or <code>-x1</code>	Composite colour table: 0 calculated/XRoar-style, 1 scanned-monitor sample values.

These options create images, not compiled sprites, and their output is not loaded with `SPRITE_LOAD` . See the next section for the current status.

Background-image output

The compiled-sprite path is the mature, tested part of PNGtoCCSB and is the recommended use described by this manual.

PNGtoCCSB 1.11 also exposes `-c1` through `-c6`, but several background formats still have known packing or output limitations. Treat this renderer as experimental and verify its byte count and picture on the target mode before using it in a project. In particular, do not use GMODE 160 through GMODE 165 as compiled-sprite targets.

For established CoCo 3 background and playfield workflows, use the dedicated `PNGtoCC3Background` and `PNGtoCC3Playfield` utilities supplied with the compiler until the PNGtoCCSB background paths have been brought to the same tested level as its sprite generator.

When `-c1` is used successfully for a standard background, the generated LOADM file has an original load address of zero. A BASIC program can add its graphics-page address as the `LOADM` offset. `-c2` and `-c3` are raw-data formats for programs or assembly projects that already know the destination address and layout.

Troubleshooting

PNGtoCCSB says the file is missing or empty

- Check spelling and capitalization.
- Put every option before the filename.
- Quote a path containing spaces.
- Run the command from the compiler folder or supply the correct relative/absolute input path.

I cannot find the generated file

Look in the folder from which PNGtoCCSB was launched, not necessarily beside a PNG supplied with a path. The output name is the PNG base name plus `.asm`, `.BIN`, or `.RAW`.

The compiler cannot open the sprite file

The `SPRITE_LOAD` path is relative to the compiler working folder. Move the `.asm` there or use a correct relative subfolder path.

The assembler reports an invalid label

Rename the source PNG using only letters, numbers, and underscores, preferably beginning with a letter, and regenerate the `.asm`. Renaming only the assembly file is not sufficient.

The sprite has the wrong colours

- Confirm that `-gN` exactly matches `GMODE N`.
- On a CoCo 1/2 four-colour mode, confirm that `-sN` matches `SCREEN 1,N`.
- On a CoCo 3, confirm that every sprite was converted with the same `CoCo3_Palette.asm`.
- Remember that analogue composite colours vary by machine, emulator settings, and display.
- Use cell-aligned positions in semigraphics modes when exact colour regions matter.

The sprite is stretched

Some low-resolution modes deliberately apply aspect correction because CoCo display cells are not square. PNGtoCCSB's reported dimensions are the actual logical footprint. GMODE 0-8 details are summarized in [Choosing the GMODE](#).

The sprite is cut off or the program crashes near an edge

The sprite runtime does not clip. Reduce X or Y so the complete generated width and height remain inside the display.

The sprite flickers

- Use `WAIT VBL` for every sprite update.
- Reserve both graphics pages.
- Finish drawing the background and then run `GCOPY 0,1` once.
- Do not place an extra `WAIT VBL` between erasing and redrawing a moving sprite.

An animated frame crashes

Use the same frame count in `-aN` and `SPRITE_LOAD`, and keep the frame value from 0 through N-1. Frame bounds are not checked at runtime.

GMODE 4 looks like large incorrect blocks on a CoCo 2B

True SG6 requires the original MC6847. The MC6847T1 in the CoCo 2B interprets those bytes differently. Use an original CoCo/MC6847 target for GMODE 4 or choose another graphics mode.

A CoCo 3 conversion cannot find `CoCo3_Palette.asm`

Generate the shared palette once with `-makepal`, then rerun the remaining conversions without it.

Quick-reference checklist

Before converting:

- ☐ The input is a 32-bit RGBA PNG.
- ☐ Transparent edges use alpha below 127.
- ☐ The filename is assembler-safe.
- ☐ The intended GMODE and colour set are known.
- ☐ Animation frames, if any, are equal-width and arranged horizontally.

Before compiling:

- ☐ The generated `.asm` is accessible from the compiler working folder.
- ☐ `SPRITE_LOAD` uses a unique number from 0 through 31.
- ☐ Its optional frame count matches `-aN`.
- ☐ The BASIC `GMODE` exactly matches PNGtoCCSB's `-gN`.
- ☐ `SCREEN 1,N` matches `-sN` where applicable.
- ☐ CoCo 3 sprites all share one palette.

Before running:

- ☐ Two graphics pages have been reserved where required.
- ☐ The completed background has been copied with `GCOPY 0,1`.
- ☐ Every update reaches `WAIT VBL`.
- ☐ The complete generated sprite footprint stays on screen.
- ☐ Multiple sprites are erased in reverse drawing order.

Command examples

Single GMODE 15 sprite using colour set 0:

```
./PNGtoCCSB -g15 -s0 HERO.png
```

Eight-frame GMODE 15 animation:

```
./PNGtoCCSB -g15 -s0 -a8 WOLF_WALK.png
```

GMODE 2 semigraphics sprite:

```
./PNGtoCCSB -g2 SG4_PLAYER.png
```

GMODE 4 sprite for the original MC6847, colour set 1:

```
./PNGtoCCSB -g4 -s1 SG6_PLAYER.png
```

CoCo 3 sprite that creates the project's palette:

```
./PNGtoCCSB -g136 -makepal PLAYER.png
```

Additional CoCo 3 sprite using that palette:

```
./PNGtoCCSB -g136 ENEMY.png
```

CoCo 3 horizontal-playfield sprite:

```
./PNGtoCCSB -g136 -scroll PLAYER.png
```

Normal background restoration is already the default, so these two commands are equivalent:

```
./PNGtoCCSB -g15 HERO.png  
./PNGtoCCSB -g15 -b1 HERO.png
```

For the complete BASIC sprite command reference and the full GMODE resolution tables, see the main **BasTo6809 Compiler User Manual**.